

Niko 2.0 — Development Notes

From the original Niko prototype to the independent compiler/VM vision

1. Starting Point — Niko 1

The original Niko project was a beginner-friendly programming language whose source looked like English instructions. It used .niko files, indentation-based blocks, a REPL, a browser IDE, a standard helper library, examples, tests, and friendly error messages.

The first engine translated Niko into Python. The browser IDE had a JavaScript engine. This was useful for rapid prototyping, but it also meant Python/JavaScript semantics could leak into Niko.

2. Why Niko 2.0 Was Needed

The goal changed from “make an English-like translator” to “build a real programming language.”

Regular-expression translation is useful for a prototype, but a growing language needs a lexer, parser, AST, semantic analysis, type checking, an intermediate representation, and explicit runtime semantics.

3. Niko 2.0 Target Architecture

The planned architecture is:

```
.niko source
-> Lexer
-> Parser
-> AST
-> Static/type analysis
-> Niko IR
-> Backend
    -> Niko VM
    -> WebAssembly
    -> Native
```

4. Alpha 1 — Compiler Foundation

We introduced the compiler-oriented structure: lexer, parser, AST and a clear separation between source syntax and execution. The old Niko engine was preserved for compatibility while Niko 2 grew alongside it.

5. Alpha 2 — Static Type System and Modules

Niko 2 gained optional type annotations such as:

```
set age: number to 25
set name: text to "Niko"

to add with a: number, b: number -> number:
    give back a + b
```

6. Alpha 3 — Niko IR and Bytecode VM

The major architectural step was the Niko IR and stack-based bytecode VM. The execution path became:

Niko source -> AST -> compiler -> Niko bytecode -> Niko VM

Niko 2 no longer needs to generate Python source to execute a program. The current VM is written in Python only as a bootstrap runtime.

7. Alpha 4 — Projects, Modules, Standard Library Foundation

The current update adds project tooling and VM-native modules.

Commands:

```
niko2 init myapp
niko2 info main.niko
niko2 run main.niko
```

8. What Niko Is Trying to Become

Niko is intended to be a general-purpose language that is easy to read but not limited by beginner-level features.

Potential application areas include education, scripting, automation, command-line tools, data processing, web services, developer tools, WebAssembly applications and eventually native software.

The differentiator should be coherent design rather than simply copying features from existing languages.

9. Future Roadmap

Alpha 5: serialized Niko IR loader, better diagnostics with columns/carets, formatter, cross-file type checking and a clearer standard-library API.

Alpha 6: generics, Option/Result-style error handling, pattern matching, dependency graph and lock file.

Alpha 7: language server, VS Code extension and debugger support.

Alpha 8: WebAssembly backend.

Alpha 9: native backend.

Niko 1 remains the compatibility/prototype engine until Niko 2 has a strong conformance suite.

10. Development Rules

Every new language feature should have a syntax decision, semantic/type behavior, runtime behavior, diagnostics, tests and documentation.

Do not add large collections of aliases. Do not make Python the hidden specification. Keep VM semantics explicit so future backends can reproduce the same language behavior.

11. Current Release

Niko 2.0 Alpha 4.

Implemented and verified:

- static type checker
- Niko IR
- bytecode compiler
- stack VM
- project manifest
- project initialization
- project information
- VM-native module loading
- standard library organization foundation
- regression suite: 26 cases, 0 failures at release verification time